Lab Exercises 10 – JavaScript (Client Storage)

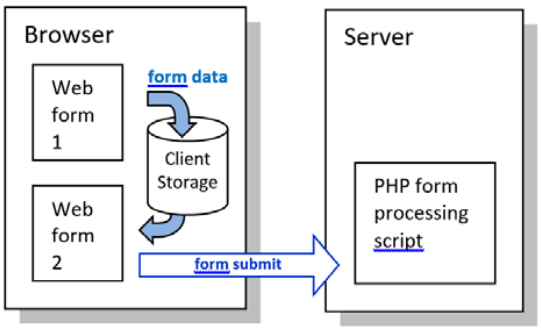
Aims

- To practice how to store the form data at client site.
- To review JavaScript functions and control structure.
- To gain the skills and knowledge to complete Assignment 2.

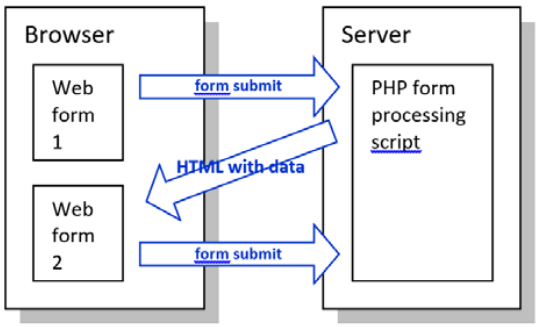
Task 1: Transferring data between HTML pages using client web storage

Sometimes we want to transfer the information entered in a form on one Web page to another Web page. For example, for the **register.html** form we used in lab 9, we want to display a confirmation page for the user to accept before the data is submitted to the Web server.

There are couple of method of doing this:



Method 1 – Using Client Storage



Method 2 – Server-side processing

Method 1 transfers data the user has entered into their Form 1 via local client storage which is managed by the browser. This data set is then used to initialize the second form when it loads. After the user has checked it (possible provide extra information) this second form is then submitted to the server.

In Method 2, Form 1 submits the user data set to a PHP script on the server. The script then used to create a new HTML form page that incorporates the data the user sent to the server. This second form then completed by the user and submitted to the server.

In Web Storage data are stored in ***key/value*** pairs. There are two new objects for storing data on the client:

- **localStorage** - stores data with no expiration date
- **sessionStorage** - stores data for one session (while the browser window /tab is open)

In this Task we will use **sessionStorage**. The way to store a key/value pair is:

```
sessionStorage.key = value;
```

For example, to store the value of an HTML element with `id = username` in JavaScript we could say:

```
sessionStorage.uname = document.getElementById("username").value;
```

or alternatively

```
sessionStorage.setItem("uname","username");
```

To retrieve a value from sessionStorage (or localStorage) and store it in a JavaScript variable we write:

```
value = sessionStorage.key;
```

For example:

```
var username = sessionStorage.uname
```

As explained above, we will send data from **register.html** to the Web page **confirm.html** via **sessionStorage**. The basic procedure is:

1. Set the form on **register.html** to link to **confirm.html** .
2. After the data on the **register.html** form have been successfully validated, write the value of the input fields or selected options to **sessionStorage**.
3. When the **confirm.html** window loads, read the values from **sessionStorage** and write these to **confirm.html** using JavaScript.

Step 1: Modify the Form (register.html)

1. In register.html, change the action attribute of the form so that it hyperlinks to **confirm.html**.

```
<form id="regForm" method="post" action="confirm.html" novalidate="novalidate">
```

Step 2: Storing User Data

In this step we will write a function that stores the values the user has entered into the form through **sessionStorage**. For now, we will working on 2 input values: **firstname** and **email**. We will call this function at the end of the **validateForm()**.

1. In **validate.js**, create a function called **storeBooking**() as shown below. As we already have the variables for **firstname** and **email** in the **validateForm() function**, we will pass these to the function as parameters.

```
function storeBooking(firstname, email){

    //get values from the parameters above and store them as sessionStorage
    attribute.

    //we use the same name for the attribute and the element id to avoid confusion


    sessionStorage.firstname = firstname;
     sessionStorage.email = email;


    //add other elements here
```

2. Extend the function to store the rest of the user's information in **sessionStorage**.
3. Invoke the **storageBooking()** function from the **validateForm()** function as shown below. We only want to do this if the variable **isAllOK** returns a true value.

```
    function validateForm(){

    "use strict";   //directive to ensure variables are declared


    var isAllOK=false;

    var firstname =
    document.getElementById("name").value; var email =
    document.getElementById("email").value;


    if ((firstname.length>0) && (email.length>0))

    {

           isAllOK=true;

    }


    if (isAllOK){

           storeBooking(firstname,email)

    }


           return isAllOK;

    }
```

4. Add the codes to trigger **validateForm()** function when the **register.html** is loaded.
5. Load **register.html** and test it.

Step 3: Create/Modify a web page to accept data

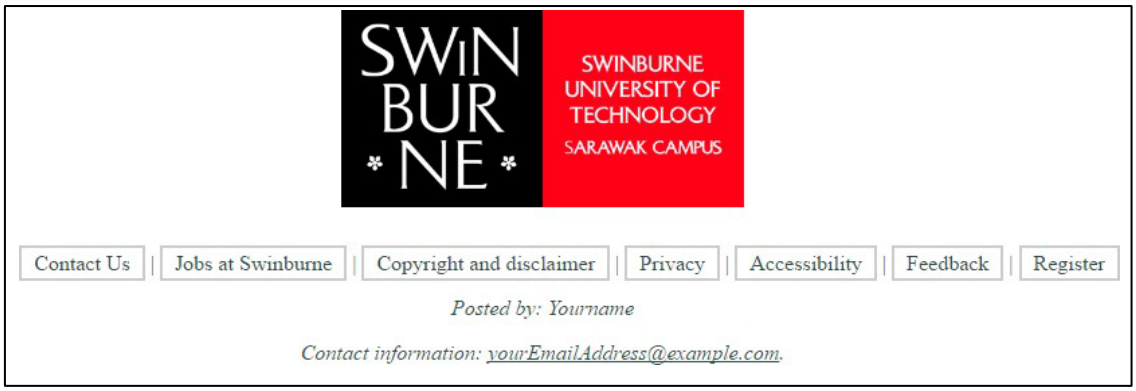
1. Now create a HTML page, named it as **confirm.html**.
2. Add the following codes to **confirm.html**.

```
<!DOCTYPE html>
<html>
 <head>
    <meta charset="utf-8" />
    <title>COS10011 Creating Web Applications</title>

    <!-- link to my CSS -->
    <link href= "styles/jefferystyle.css" rel="stylesheet" />

 </head>
```

3. What we are doing here is creating a simple HTML file based on the design we have done in the previous lab exercise. The reason for doing this is to maintain the consistency throughout all the pages.
4. Please note that your codes might different from the codes above. Please modify them accordingly so they will be the same as yours.
5. View your **confirm.html** page and it should look something as below:



6. Next, create a new file named **confirm.js**.

7. In the **<head>** tag, create a hyperlink which links to **confirm.js**.

```
    <!-- confirm javascript -->
    <script src="js/confirm.js"></script>
```

8. Next, create a basic page to display all the user's inputs from **register.html**. In this example, it only stores two values: Name and Email address.

```
<form id="bookform">
    <fieldset>
        <legend>User Details</legend>
        <p>Your Name: <span id="confirm_name"></span></p>
        <p>Your Email: <span id="confirm_email"></span></p>
        <!--Create the rest of the data output here -->

        <input type="submit" value="Confirm Booking" />
        <input type="button" value="Cancel" id="cancelButton"
        onclick="cancelBooking()">
    </fieldset>
</form>
```

9. Next, continue with all the data that need to be stored, such as **Your Tutor, Your unit, Issue, and Description**.
10. Let's working on **confirm.js**. Add the following codes to your js file.

```
"use strict";
/*get variables from form and check rules*/
function validate(){
	var errMsg = ""; /* stores the error message */
	var result = true; /* assumes no errors */
	if(result){
		alert("Booking Successfully");
	}else{
		alert("Booking Failed");
	}
return result; //if false the information will not be sent to the server
}

function getBooking(){
	//if sessionStorage for username is not empty
	if(sessionStorage.firstname != undefined){
	//confirmation text
	document.getElementById("confirm_name").textContent =
sessionStorage.firstname;
	document.getElementById("confirm_email").textContent =sessionStorage.email;
	}
}

function cancelBooking(){
	window.location = "register.html";
}

function init () {
	//call the getBooking() function.
	getBooking();
	// Link the variable to the HTML element
	var bookForm = document.getElementById("bookform");
	var cancelBooking = document.getElementById("cancelButton");
	//check which button is being click.
	if(bookForm){
		bookForm.onsubmit = validate;
	}else{
		cancelBooking.onclick = cancelBooking();
	}
}
window.onload = init;
```

11. Let's try to test your registration form. Once you click Register, it will be directed to the **confirm.html** and you should see the result as below:

